Task 1: Convolutional-Recurrent Architecture for Art Classification

Objective: Build a CNN-RNN model to classify paintings by Style, Artist, and Genre using the WikiArt dataset.

Strategy

Why CNN-RNN?

Paintings contain both local patterns (brushstrokes, textures, color palettes) and global compositional structure (arrangement of subjects, spatial relationships). A pure CNN captures local-to-global features hierarchically, but a recurrent layer on top of spatial feature maps can explicitly model sequential dependencies across image regions — similar to how an art expert scans different parts of a painting to form a judgment.

Architecture:

CNN Backbone (ResNet-50, pretrained on ImageNet): Extracts rich spatial feature maps

Spatial Patch Sequence: The CNN feature map is treated as a sequence of spatial patches

Bidirectional LSTM: Processes the patch sequence to capture inter-region dependencies

Attention Mechanism: Learns which spatial regions are most discriminative for each task

Multi-Task Heads: Separate FC classifiers for Style (27), Artist (129), Genre (11)

Evaluation Metrics:

Top-1 & Top-5 Accuracy (especially for Artist with 129 classes)

Macro & Weighted F1-Score (handles class imbalance)

Per-class Precision & Recall

Confusion Matrices

Outlier Detection via confidence analysis and embedding visualization

This notebook is run in google colab.

1. Setup & Installation

```
# Install dependencies
import subprocess, sys
try:
    import datasets
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-
q",
        "datasets", "faiss-cpu", "umap-learn"])

import os, sys, time, random, warnings
import numpy as np
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter, defaultdict
from tqdm.auto import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, Subset
from torchvision import transforms, models
from PIL import Image

from sklearn.metrics import (accuracy_score, f1_score,
classification_report,
                             confusion_matrix)
from sklearn.manifold import TSNE

warnings.filterwarnings('ignore')
sns.set_palette("husl")

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# Configuration
DEMO_MODE = True  # Set to False for full training

IMG_SIZE = 224
BATCH_SIZE = 32
NUM_WORKERS = 2
LEARNING_RATE = 1e-4
WEIGHT_DECAY = 1e-4
NUM_EPOCHS = 5 if DEMO_MODE else 30
MAX_SAMPLES = 2000 if DEMO_MODE else None  # None = use all

print(f" ArtExtract Task 1 — CNN-RNN Art Classifier")
print(f"   Mode: {'DEMO (subset)' if DEMO_MODE else 'FULL'}")
print(f"   Epochs: {NUM_EPOCHS}")
```

```
 ArtExtract Task 1 — CNN-RNN Art Classifier
   Mode: DEMO (subset)
   Epochs: 5
```

```python
# Device setup
def get_device():
    if torch.cuda.is_available():
```

```
        device = torch.device('cuda')
        print(f" Using GPU: {torch.cuda.get_device_name(0)}")
    elif hasattr(torch.backends, 'mps') and
torch.backends.mps.is_available():
        device = torch.device('mps')
        print(" Using Apple MPS")
    else:
        device = torch.device('cpu')
        print(" Using CPU — training will be slow")
    return device

device = get_device()
```

```
 Using CPU — training will be slow
```

2. Data Loading — WikiArt Dataset (Memory-Efficient)

We use the HuggingFace `huggan/wikiart` dataset which provides streaming access to WikiArt images without downloading the full 25GB dataset.

```
from datasets import load_dataset

print(" Loading WikiArt dataset from HuggingFace (streaming)...")
print("   This avoids downloading the full 25GB dataset.")

# Load dataset with streaming for memory efficiency
try:
    hf_dataset = load_dataset("huggan/wikiart", split="train",
trust_remote_code=True)
    print(f" Loaded {len(hf_dataset)} samples")
except Exception as e:
    print(f" HuggingFace load failed: {e}")
    print("   Trying alternative: huggan/wikiart...")
    hf_dataset = load_dataset("huggan/wikiart", split="train")

# Inspect the dataset features
print(f"\n Dataset Features:")
print(hf_dataset.features)
```

```
`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'huggan/wikiart' isn't
based on a loading script and remove `trust_remote_code`.
If the dataset is based on a loading script, please ask the dataset
author to remove it and convert it to a standard format like Parquet.
ERROR:datasets.load:`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'huggan/wikiart' isn't
based on a loading script and remove `trust_remote_code`.
If the dataset is based on a loading script, please ask the dataset
author to remove it and convert it to a standard format like Parquet.
```

```
Loading WikiArt dataset from HuggingFace (streaming)...
  This avoids downloading the full 25GB dataset.
```


```
{"model_id":"42dbdf95fc95450eb04690f204f32ac4","version_major":2,"version_minor":0}

{"model_id":"0b9dc8659b654631b42dad767f30adf2","version_major":2,"version_minor":0}

{"model_id":"23575067d6fe4c52a47871f33b2a7ca0","version_major":2,"version_minor":0}

{"model_id":"594fff43b09647d0bc7c114dd59a7c44","version_major":2,"version_minor":0}

{"model_id":"218c756e250748f291c0db75500304ed","version_major":2,"version_minor":0}

{"model_id":"25fa5d7c39464f0ea29ed0e2c54cb58b","version_major":2,"version_minor":0}

{"model_id":"7818d57b3a41413eb5efbcf223e6bb61","version_major":2,"version_minor":0}

{"model_id":"cf528c6652f04e7c8e69d0c4e9e1ebd5","version_major":2,"version_minor":0}

{"model_id":"0a19b7b0232d492bbc3b594b1a489f83","version_major":2,"version_minor":0}

{"model_id":"a9179cbaf7e74f2f9eb9ea80bcde841e","version_major":2,"version_minor":0}
```


```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"a83b4aad450443008fa57137187c8c4e","version_major":2,"version_minor":0}

{"model_id":"2e35a0bdd3f44013adfe62d5e0500544","version_major":2,"version_minor":0}

{"model_id":"4b4c5e2742e0440c846588eba06287a2","version_major":2,"version_minor":0}

{"model_id":"53f4c3c803b34658a1964a1c8afa82c3","version_major":2,"version_minor":0}
```

{"model_id":"47fdc2edae10443082b6e07eb12075a3","version_major":2,"version_minor":0}

{"model_id":"dee0ed90af394e6c920950e251771b81","version_major":2,"version_minor":0}

{"model_id":"eb8ae938a76745c39644f13c45b46121","version_major":2,"version_minor":0}

{"model_id":"5267016f50be46378d5fbaaa062e954f","version_major":2,"version_minor":0}

{"model_id":"510c9e48c35547e88ce84f9ba084a9cd","version_major":2,"version_minor":0}

{"model_id":"a11f13363fcb4362a5313070542eed97","version_major":2,"version_minor":0}

{"model_id":"429635c1da5640aa85c8ad32d85a623e","version_major":2,"version_minor":0}

{"model_id":"225f8b3b11194e059a7180f9d057e9e2","version_major":2,"version_minor":0}

{"model_id":"fc00b51b5c36402abdafc888a27def7a","version_major":2,"version_minor":0}

{"model_id":"98a17ac0c96c460886eae1b32e4fc52e","version_major":2,"version_minor":0}

{"model_id":"ed767b947872448e8483020a5da3678c","version_major":2,"version_minor":0}

{"model_id":"b85defd4521f427f86ced7f5458b175c","version_major":2,"version_minor":0}

{"model_id":"5ee3e9e7c03b42dab51bc5875f7e5a92","version_major":2,"version_minor":0}

{"model_id":"18656a0b28f14f2aa5b9e1fb415fac08","version_major":2,"version_minor":0}

{"model_id":"462d664fb7e94664896ed404e0aa8c8e","version_major":2,"version_minor":0}

{"model_id":"a6c6aa8a29784d2ca1388e6e7338fa14","version_major":2,"version_minor":0}

{"model_id":"b16af2d58ffb49e5844fec40b3c4d2a9","version_major":2,"version_minor":0}

{"model_id":"f21907dcbfcc45dcb94b591854d323ed","version_major":2,"version_minor":0}

{"model_id":"422b675ab3cd42afaaec65e53c05257b","version_major":2,"version_minor":0}

{"model_id":"c2e48bee2d244796bfb0be0ded2ac687","version_major":2,"version_minor":0}

{"model_id":"d54c40cc85e248c5acec0be7f61423ca","version_major":2,"version_minor":0}

{"model_id":"0c4a4a60fa4e43a7849137cc43400bfd","version_major":2,"version_minor":0}

{"model_id":"59dbe6a8733248cfabeaf943c4542f73","version_major":2,"version_minor":0}

{"model_id":"0267abd35490484b81040ff904d06843","version_major":2,"version_minor":0}

{"model_id":"e3a654d56efa488a9a729712b8e45e43","version_major":2,"version_minor":0}

{"model_id":"9e21fff9f10e4693a8f17ae0d3948edd","version_major":2,"version_minor":0}

{"model_id":"6e0c291aa01e4eebba3b07b187be8889","version_major":2,"version_minor":0}

{"model_id":"2eae4a8b592449f28dde4aec2ace4c41","version_major":2,"version_minor":0}

{"model_id":"49c6dde404404941ac4831cd364c4b2a","version_major":2,"version_minor":0}

{"model_id":"10000ee057b1410488aab56b84456e18","version_major":2,"version_minor":0}

{"model_id":"1fe9a43234a84c6fa92d78454455e7a7","version_major":2,"version_minor":0}

{"model_id":"9db66def063249b49b2cfdaf93bbf5b1","version_major":2,"version_minor":0}

{"model_id":"86acf0dc061c42358bd2c75910969eca","version_major":2,"version_minor":0}

{"model_id":"c519dade28764dc084b924bad93e3aba","version_major":2,"version_minor":0}

{"model_id":"1dfa22b79eb446ccb229501062018f4b","version_major":2,"version_minor":0}

{"model_id":"a7419b7d14c847a8aca47c1dba26df7f","version_major":2,"version_minor":0}

{"model_id":"617fec27aac446ae9ee0781276c37dba","version_major":2,"version_minor":0}

{"model_id":"c97e4241707f4d699c54fd319a0de067","version_major":2,"version_minor":0}

{"model_id":"ae7b0347863e405db6555f1eacb89f72","version_major":2,"version_minor":0}

{"model_id":"c910e4e5b4a74ff9b2c5aa8b561d42f5","version_major":2,"version_minor":0}

{"model_id":"12448ef9a6bd4ce9b545f3157211b023","version_major":2,"version_minor":0}

{"model_id":"a9eb000acc45436799ecf7efee5e5c74","version_major":2,"version_minor":0}

{"model_id":"c625f48540574f20a46240c3f406bc51","version_major":2,"version_minor":0}

{"model_id":"2b2b556438fe4706805dd2a1791774bf","version_major":2,"version_minor":0}

{"model_id":"fddd8df04c4941439687e727e65c7947","version_major":2,"version_minor":0}

{"model_id":"2c6751d84fa3488aa1510a4c89e12fc6","version_major":2,"version_minor":0}

{"model_id":"c288962c93904bd69cdb483931b23b7f","version_major":2,"version_minor":0}

{"model_id":"7937080fe4a84f9f9b20c6ae956dd7fe","version_major":2,"version_minor":0}

{"model_id":"6fc4a47bbfb04159baf818f03d403f32","version_major":2,"version_minor":0}

{"model_id":"592d799f304e4bc9a504a719c8e8aa0b","version_major":2,"version_minor":0}

{"model_id":"16c9da53da6749aaa0c76dd5980beacf","version_major":2,"version_minor":0}

{"model_id":"d6164cb14fa54e6e814b47c7e7b91a2d","version_major":2,"version_minor":0}

{"model_id":"d3769ea907364a659eef7a0887b12843","version_major":2,"version_minor":0}

{"model_id":"3ec8cfb759144505b4d95bc0443f4f8f","version_major":2,"version_minor":0}

{"model_id":"d6e43245407649e0934b7bc3102d9030","version_major":2,"version_minor":0}

{"model_id":"284e60d6a3f144a98696846e4022c1d8","version_major":2,"version_minor":0}

{"model_id":"d15c7a238f414b709f3a98a2fb3efba1","version_major":2,"version_minor":0}

{"model_id":"13c332e387e346b5b6877268bfc9b95e","version_major":2,"version_minor":0}

{"model_id":"9dd2c4bf1ebb4bf68b6e385bbc08ec5d","version_major":2,"version_minor":0}

{"model_id":"3819e2608b344446b389c3583be24032","version_major":2,"version_minor":0}

{"model_id":"0dcbf298d6f74cf0866fc226c3194b29","version_major":2,"version_minor":0}

{"model_id":"04577e1a596e4b648c0ae4a1a0a47fd4","version_major":2,"version_minor":0}

{"model_id":"735c41c70ea8423a9bc0d5e4d101141e","version_major":2,"version_minor":0}

{"model_id":"44ebe9ed059d429fb4ac500af769cdfb","version_major":2,"version_minor":0}

```
Loaded 81444 samples

Dataset Features:
{'image': Image(mode=None, decode=True), 'artist':
ClassLabel(names=['Unknown Artist', 'boris-kustodiev', 'camille-
pissarro', 'childe-hassam', 'claude-monet', 'edgar-degas', 'eugene-
boudin', 'gustave-dore', 'ilya-repin', 'ivan-aivazovsky', 'ivan-
shishkin', 'john-singer-sargent', 'marc-chagall', 'martiros-saryan',
'nicholas-roerich', 'pablo-picasso', 'paul-cezanne', 'pierre-auguste-
renoir', 'pyotr-konchalovsky', 'raphael-kirchner', 'rembrandt',
'salvador-dali', 'vincent-van-gogh', 'hieronymus-bosch', 'leonardo-da-
vinci', 'albrecht-durer', 'edouard-cortes', 'sam-francis', 'juan-
gris', 'lucas-cranach-the-elder', 'paul-gauguin', 'konstantin-
makovsky', 'egon-schiele', 'thomas-eakins', 'gustave-moreau',
'francisco-goya', 'edvard-munch', 'henri-matisse', 'fra-angelico',
'maxime-maufra', 'jan-matejko', 'mstislav-dobuzhinsky', 'alfred-
sisley', 'mary-cassatt', 'gustave-loiseau', 'fernando-botero',
'zinaida-serebriakova', 'georges-seurat', 'isaac-levitan', 'joaquã\
xadn-sorolla', 'jacek-malczewski', 'berthe-morisot', 'andy-warhol',
'arkhip-kuindzhi', 'niko-pirosmani', 'james-tissot', 'vasily-polenov',
'valentin-serov', 'pietro-perugino', 'pierre-bonnard', 'ferdinand-
hodler', 'bartolome-esteban-murillo', 'giovanni-boldini', 'henri-
```

```
martin', 'gustav-klimt', 'vasily-perov', 'odilon-redon', 'tintoretto',
'gene-davis', 'raphael', 'john-henry-twachtman', 'henri-de-toulouse-
lautrec', 'antoine-blanchard', 'david-burliuk', 'camille-corot',
'konstantin-korovin', 'ivan-bilibin', 'titian', 'maurice-prendergast',
'edouard-manet', 'peter-paul-rubens', 'aubrey-beardsley', 'paolo-
veronese', 'joshua-reynolds', 'kuzma-petrov-vodkin', 'gustave-
caillebotte', 'lucian-freud', 'michelangelo', 'dante-gabriel-
rossetti', 'felix-vallotton', 'nikolay-bogdanov-belsky', 'georges-
braque', 'vasily-surikov', 'fernand-leger', 'konstantin-somov',
'katsushika-hokusai', 'sir-lawrence-alma-tadema', 'vasily-
vereshchagin', 'ernst-ludwig-kirchner', 'mikhail-vrubel', 'orest-
kiprensky', 'william-merritt-chase', 'aleksey-savrasov', 'hans-
memling', 'amedeo-modigliani', 'ivan-kramskoy', 'utagawa-kuniyoshi',
'gustave-courbet', 'william-turner', 'theo-van-rysselberghe', 'joseph-
wright', 'edward-burne-jones', 'koloman-moser', 'viktor-vasnetsov',
'anthony-van-dyck', 'raoul-dufy', 'frans-hals', 'hans-holbein-the-
younger', 'ilya-mashkov', 'henri-fantin-latour', 'm.c.-escher', 'el-
greco', 'mikalojus-ciurlionis', 'james-mcneill-whistler', 'karl-
bryullov', 'jacob-jordaens', 'thomas-gainsborough', 'eugene-
delacroix', 'canaletto']), 'genre':
ClassLabel(names=['abstract_painting', 'cityscape', 'genre_painting',
'illustration', 'landscape', 'nude_painting', 'portrait',
'religious_painting', 'sketch_and_study', 'still_life', 'Unknown
Genre']), 'style': ClassLabel(names=['Abstract_Expressionism',
'Action_painting', 'Analytical_Cubism', 'Art_Nouveau', 'Baroque',
'Color_Field_Painting', 'Contemporary_Realism', 'Cubism',
'Early_Renaissance', 'Expressionism', 'Fauvism', 'High_Renaissance',
'Impressionism', 'Mannerism_Late_Renaissance', 'Minimalism',
'Naive_Art_Primitivism', 'New_Realism', 'Northern_Renaissance',
'Pointillism', 'Pop_Art', 'Post_Impressionism', 'Realism', 'Rococo',
'Romanticism', 'Symbolism', 'Synthetic_Cubism', 'Ukiyo_e'])}

# Explore the label space
print("\n Label Information:")

# Get label names from features
if hasattr(hf_dataset.features.get('style', None), 'names'):
    STYLE_NAMES = hf_dataset.features['style'].names
    print(f"  Styles ({len(STYLE_NAMES)}): {STYLE_NAMES[:5]}...")
else:
    # Fallback: extract unique values
    styles = set()
    for i, sample in enumerate(hf_dataset):
        styles.add(sample.get('style', -1))
        if i > 5000: break
    STYLE_NAMES = [f"Style_{i}" for i in range(max(styles)+1)]
    print(f"  Styles: {len(STYLE_NAMES)} classes found")

if hasattr(hf_dataset.features.get('artist', None), 'names'):
    ARTIST_NAMES = hf_dataset.features['artist'].names
```

```
    print(f"  Artists ({len(ARTIST_NAMES)}): {ARTIST_NAMES[:5]}...")
else:
    ARTIST_NAMES = None
    print("  Artists: extracting from data...")

if hasattr(hf_dataset.features.get('genre', None), 'names'):
    GENRE_NAMES = hf_dataset.features['genre'].names
    print(f"  Genres ({len(GENRE_NAMES)}): {GENRE_NAMES[:5]}...")
else:
    GENRE_NAMES = None
    print("  Genres: extracting from data...")

NUM_STYLES = len(STYLE_NAMES) if STYLE_NAMES else 27
NUM_ARTISTS = len(ARTIST_NAMES) if ARTIST_NAMES else 129
NUM_GENRES = len(GENRE_NAMES) if GENRE_NAMES else 11

print(f"\n  → {NUM_STYLES} styles, {NUM_ARTISTS} artists, {NUM_GENRES}
genres")
```

```
 Label Information:
  Styles (27): ['Abstract_Expressionism', 'Action_painting',
'Analytical_Cubism', 'Art_Nouveau', 'Baroque']...
  Artists (129): ['Unknown Artist', 'boris-kustodiev', 'camille-
pissarro', 'childe-hassam', 'claude-monet']...
  Genres (11): ['abstract_painting', 'cityscape', 'genre_painting',
'illustration', 'landscape']...

  → 27 styles, 129 artists, 11 genres
```

```
# Subsample for demo mode
if MAX_SAMPLES and len(hf_dataset) > MAX_SAMPLES:
    indices = random.sample(range(len(hf_dataset)), MAX_SAMPLES)
    hf_dataset = hf_dataset.select(indices)
    print(f" Subsampled to {MAX_SAMPLES} images for demo mode")

# Train/Val split (80/20)
split = hf_dataset.train_test_split(test_size=0.2, seed=SEED)
train_hf = split['train']
val_hf = split['test']
print(f" Train: {len(train_hf)} | Val: {len(val_hf)}")
```

```
 Subsampled to 2000 images for demo mode
 Train: 1600 | Val: 400
```

3. PyTorch Dataset & Transforms

```
# Data augmentation and normalization
train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE + 32, IMG_SIZE + 32)),
```

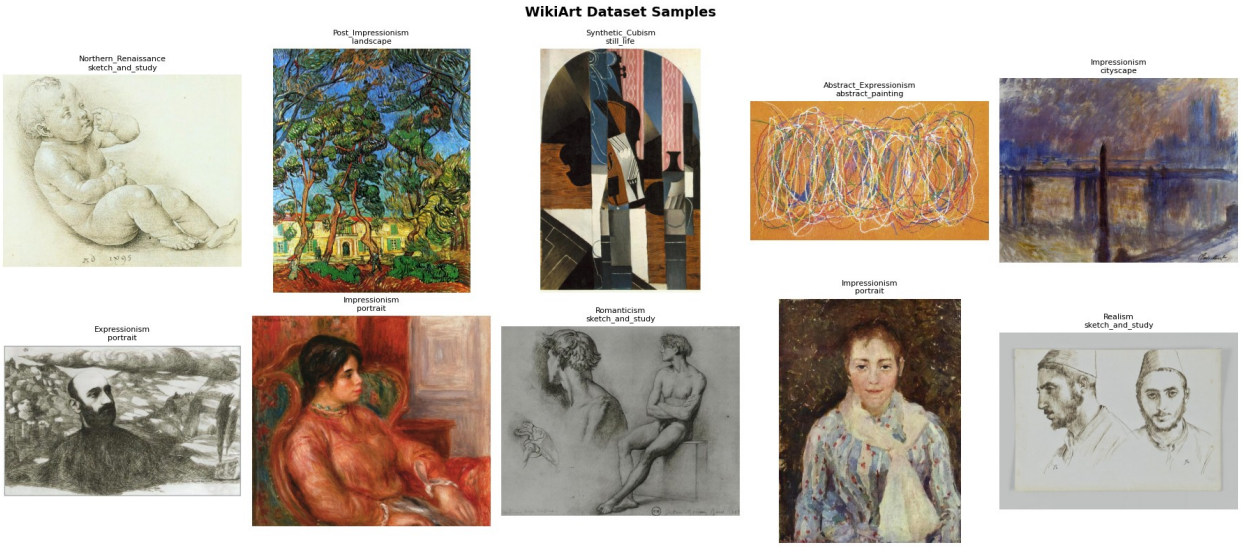
```
    transforms.RandomCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]),
])

val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]),
])


class WikiArtDataset(Dataset):
    """PyTorch Dataset wrapping HuggingFace WikiArt data."""

    def __init__(self, hf_data, transform=None):
        self.data = hf_data
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        image = sample['image']

        # Ensure RGB
        if image.mode != 'RGB':
            image = image.convert('RGB')

        if self.transform:
            image = self.transform(image)

        style = sample.get('style', 0)
        artist = sample.get('artist', 0)
        genre = sample.get('genre', 0)

        return image, {
            'style': torch.tensor(style, dtype=torch.long),
            'artist': torch.tensor(artist, dtype=torch.long),
            'genre': torch.tensor(genre, dtype=torch.long),
        }


# Create datasets and dataloaders
```

```
train_dataset = WikiArtDataset(train_hf, transform=train_transform)
val_dataset = WikiArtDataset(val_hf, transform=val_transform)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True,
                          num_workers=NUM_WORKERS, pin_memory=True,
drop_last=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False,
                        num_workers=NUM_WORKERS, pin_memory=True)

print(f" DataLoaders ready: {len(train_loader)} train batches,
{len(val_loader)} val batches")
```

```
 DataLoaders ready: 50 train batches, 13 val batches
```

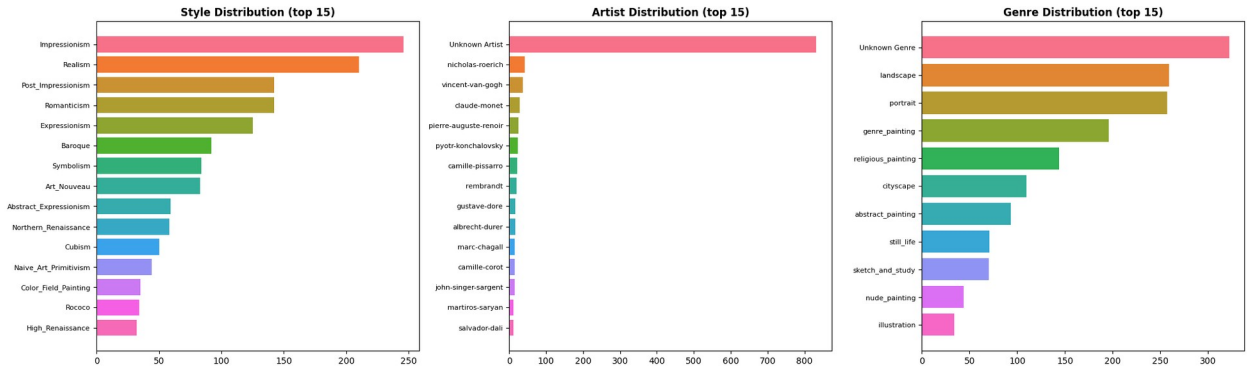
```
# Visualize sample images
print(" Sample images from the dataset:")
fig, axes = plt.subplots(2, 5, figsize=(18, 8))
for i, ax in enumerate(axes.flatten()):
    sample = train_hf[i]
    img = sample['image']
    if img.mode != 'RGB':
        img = img.convert('RGB')
    ax.imshow(img)

    s = STYLE_NAMES[sample['style']] if STYLE_NAMES and
sample['style'] < len(STYLE_NAMES) else f"S{sample['style']}"
    g = GENRE_NAMES[sample['genre']] if GENRE_NAMES and
sample['genre'] < len(GENRE_NAMES) else f"G{sample['genre']}"
    ax.set_title(f"{s}\n{g}", fontsize=8)
    ax.axis('off')

plt.suptitle("WikiArt Dataset Samples", fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.show()
```

```
 Sample images from the dataset:
```




```
# Class distribution analysis
print(" Class Distribution Analysis:")

fig, axes = plt.subplots(1, 3, figsize=(20, 6))

for idx, (task, names) in enumerate([('style', STYLE_NAMES),
('artist', ARTIST_NAMES), ('genre', GENRE_NAMES)]):
    labels = [train_hf[i][task] for i in range(len(train_hf))]
    counter = Counter(labels)
    top = counter.most_common(15)

    task_names = [names[c] if names and c < len(names) else str(c) for
c, _ in top]
    counts = [cnt for _, cnt in top]

    axes[idx].barh(range(len(task_names)), counts,
color=sns.color_palette("husl", len(task_names)))
    axes[idx].set_yticks(range(len(task_names)))
    axes[idx].set_yticklabels(task_names, fontsize=8)
    axes[idx].set_title(f"{task.title()} Distribution (top 15)",
fontweight='bold')
    axes[idx].invert_yaxis()

plt.tight_layout()
plt.show()
```

```
 Class Distribution Analysis:
```



4. Model Architecture — CNN-RNN with Attention

The model uses:

1. **ResNet-50** (pretrained) as the CNN backbone to extract spatial features
2. The feature map is reshaped into a **sequence of spatial patches**
3. A **Bidirectional LSTM** processes this sequence
4. An **Attention mechanism** weights the LSTM outputs
5. **Multi-task classification heads** predict Style, Artist, and Genre

```
class Attention(nn.Module):
    """Additive attention mechanism over a sequence."""

    def __init__(self, hidden_dim):
        super().__init__()
        self.attn = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.Tanh(),
            nn.Linear(hidden_dim // 2, 1)
        )

    def forward(self, lstm_output):
        # lstm_output: (batch, seq_len, hidden_dim)
        weights = self.attn(lstm_output)          # (batch, seq_len,
1)
        weights = F.softmax(weights, dim=1)       # normalize over
sequence
        context = (lstm_output * weights).sum(dim=1)  # (batch,
hidden_dim)
        return context, weights.squeeze(-1)


class ArtCNNRNN(nn.Module):
    """
    CNN-RNN model for multi-task art classification.

    Architecture:
        Image → ResNet-50 (feature maps) → reshape to sequence
```

```
        → BiLSTM → Attention → shared embedding
        → Style head / Artist head / Genre head
    """

    def __init__(self, num_styles=27, num_artists=129, num_genres=11,
                 lstm_hidden=256, lstm_layers=2, dropout=0.3):
        super().__init__()

        # CNN Backbone: ResNet-50 (remove the final FC + avgpool)
        resnet =
models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
        self.cnn = nn.Sequential(*list(resnet.children())[:-2])  #
Output: (B, 2048, 7, 7)

        # Freeze early CNN layers for efficiency
        for param in list(self.cnn.parameters())[:-20]:
            param.requires_grad = False

        self.cnn_dim = 2048  # ResNet-50 feature dim

        # Project CNN features to LSTM input dim
        self.feature_proj = nn.Sequential(
            nn.Linear(self.cnn_dim, lstm_hidden * 2),
            nn.LayerNorm(lstm_hidden * 2),
            nn.ReLU(),
            nn.Dropout(dropout),
        )

        # Bidirectional LSTM
        self.lstm = nn.LSTM(
            input_size=lstm_hidden * 2,
            hidden_size=lstm_hidden,
            num_layers=lstm_layers,
            batch_first=True,
            bidirectional=True,
            dropout=dropout if lstm_layers > 1 else 0,
        )

        # Attention
        self.attention = Attention(lstm_hidden * 2)  # *2 for
bidirectional

        # Shared embedding layer
        self.shared_fc = nn.Sequential(
            nn.Linear(lstm_hidden * 2, 512),
            nn.ReLU(),
            nn.Dropout(dropout),
        )

        # Task-specific classification heads
```

```
        self.style_head = nn.Sequential(
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_styles)
        )

        self.artist_head = nn.Sequential(
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_artists)
        )

        self.genre_head = nn.Sequential(
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_genres)
        )

    def forward(self, x, return_embedding=False):
        batch_size = x.size(0)

        # CNN: extract spatial feature maps
        features = self.cnn(x)  # (B, 2048, 7, 7)

        # Reshape to sequence: (B, 49, 2048)
        B, C, H, W = features.shape
        features = features.view(B, C, H * W).permute(0, 2, 1)  # (B,
49, 2048)

        # Project features
        features = self.feature_proj(features)  # (B, 49,
lstm_hidden*2)

        # BiLSTM over spatial sequence
        lstm_out, _ = self.lstm(features)  # (B, 49, lstm_hidden*2)

        # Attention pooling
        context, attn_weights = self.attention(lstm_out)  # (B,
lstm_hidden*2)

        # Shared embedding
        embedding = self.shared_fc(context)  # (B, 512)

        # Task predictions
        style_logits = self.style_head(embedding)
        artist_logits = self.artist_head(embedding)
        genre_logits = self.genre_head(embedding)
```

```
        output = {
            'style': style_logits,
            'artist': artist_logits,
            'genre': genre_logits,
            'attn_weights': attn_weights,
        }

        if return_embedding:
            output['embedding'] = embedding

        return output


# Instantiate model
model = ArtCNNRNN(
    num_styles=NUM_STYLES,
    num_artists=NUM_ARTISTS,
    num_genres=NUM_GENRES,
).to(device)

# Print model summary
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(f"\n ArtCNNRNN Model")
print(f"   Total params:     {total:>12,}")
print(f"   Trainable params: {trainable:>12,}")

# Verify with dummy input
dummy = torch.randn(2, 3, IMG_SIZE, IMG_SIZE).to(device)
out = model(dummy, return_embedding=True)
print(f"\n   Output shapes:")
print(f"   Style:     {out['style'].shape}")
print(f"   Artist:    {out['artist'].shape}")
print(f"   Genre:     {out['genre'].shape}")
print(f"   Embedding: {out['embedding'].shape}")
print(f"   Attention: {out['attn_weights'].shape}")
```

```
Downloading: "https://download.pytorch.org/models/resnet50-
0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-
0676ba61.pth

100%|██████████| 97.8M/97.8M [00:00<00:00, 116MB/s]


 ArtCNNRNN Model
   Total params:       28,543,208
   Trainable params:   13,964,456

   Output shapes:
```

```
   Style:     torch.Size([2, 27])
   Artist:    torch.Size([2, 129])
   Genre:     torch.Size([2, 11])
   Embedding: torch.Size([2, 512])
   Attention: torch.Size([2, 49])
```

5. Training Loop

```
# Loss and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(filter(lambda p: p.requires_grad,
model.parameters()),
                        lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=NUM_EPOCHS)

# Task weights for multi-task loss
TASK_WEIGHTS = {'style': 1.0, 'artist': 0.5, 'genre': 1.0}

def train_one_epoch(model, loader, optimizer, device):
    model.train()
    total_loss = 0
    task_correct = {'style': 0, 'artist': 0, 'genre': 0}
    n_samples = 0

    pbar = tqdm(loader, desc="Training", leave=False)
    for images, labels in pbar:
        images = images.to(device)
        labels = {k: v.to(device) for k, v in labels.items()}

        optimizer.zero_grad()
        outputs = model(images)

        # Multi-task loss
        loss = sum(TASK_WEIGHTS[task] * criterion(outputs[task],
labels[task])
                   for task in ['style', 'artist', 'genre'])

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
        optimizer.step()

        total_loss += loss.item() * images.size(0)
        n_samples += images.size(0)

        for task in ['style', 'artist', 'genre']:
            preds = outputs[task].argmax(dim=1)
            task_correct[task] += (preds == labels[task]).sum().item()
```

```
        pbar.set_postfix(loss=loss.item())

    avg_loss = total_loss / n_samples
    accs = {t: task_correct[t] / n_samples for t in task_correct}
    return avg_loss, accs


@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    total_loss = 0
    task_correct = {'style': 0, 'artist': 0, 'genre': 0}
    all_preds = {'style': [], 'artist': [], 'genre': []}
    all_labels = {'style': [], 'artist': [], 'genre': []}
    all_logits = {'style': [], 'artist': [], 'genre': []}
    all_embeddings = []
    n_samples = 0

    for images, labels in tqdm(loader, desc="Evaluating",
leave=False):
        images = images.to(device)
        labels_dev = {k: v.to(device) for k, v in labels.items()}

        outputs = model(images, return_embedding=True)

        loss = sum(TASK_WEIGHTS[task] * criterion(outputs[task],
labels_dev[task])
                   for task in ['style', 'artist', 'genre'])

        total_loss += loss.item() * images.size(0)
        n_samples += images.size(0)

        for task in ['style', 'artist', 'genre']:
            preds = outputs[task].argmax(dim=1)
            task_correct[task] += (preds ==
labels_dev[task]).sum().item()
            all_preds[task].extend(preds.cpu().numpy())
            all_labels[task].extend(labels[task].numpy())
            all_logits[task].append(outputs[task].cpu().numpy())

        all_embeddings.append(outputs['embedding'].cpu().numpy())

    avg_loss = total_loss / n_samples
    accs = {t: task_correct[t] / n_samples for t in task_correct}

    for task in all_logits:
        all_logits[task] = np.concatenate(all_logits[task])
    all_embeddings = np.concatenate(all_embeddings)
```

```
    return avg_loss, accs, all_preds, all_labels, all_logits,
all_embeddings

# Training
print(" Starting Training...")
print(f"   Epochs: {NUM_EPOCHS} | Batch Size: {BATCH_SIZE} | LR:
{LEARNING_RATE}")
print("=" * 70)

history = {'train_loss': [], 'val_loss': [],
           'train_acc': [], 'val_acc': []}
best_val_loss = float('inf')

for epoch in range(NUM_EPOCHS):
    t0 = time.time()

    train_loss, train_accs = train_one_epoch(model, train_loader,
optimizer, device)
    val_loss, val_accs, _, _, _, _ = evaluate(model, val_loader,
device)
    scheduler.step()

    elapsed = time.time() - t0

    # Track average accuracy across tasks
    train_avg_acc = np.mean(list(train_accs.values()))
    val_avg_acc = np.mean(list(val_accs.values()))

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_avg_acc)
    history['val_acc'].append(val_avg_acc)

    # Save best model
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model.pth')
        marker = "  best"
    else:
        marker = ""

    print(f"Epoch {epoch+1:2d}/{NUM_EPOCHS} ({elapsed:.0f}s) | "
          f"Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} |
"
          f"Style: {val_accs['style']:.3f} | Artist:
{val_accs['artist']:.3f} | "
          f"Genre: {val_accs['genre']:.3f}{marker}")

print("\n Training complete!")
```

```
 Starting Training...
   Epochs: 5 | Batch Size: 32 | LR: 0.0001
==========================================================================

{"model_id":"93785f6440924d2e9d2f2366548fe3af","version_major":2,"vers
ion_minor":0}

{"model_id":"7fd4ddd1957e4fd3bdd11a81df2d5681","version_major":2,"vers
ion_minor":0}

Epoch  1/5 (715s) | Train Loss: 7.4164 | Val Loss: 6.5219 | Style:
0.193 | Artist: 0.522 | Genre: 0.158  best

{"model_id":"efef18591d2f45ca9d81a5a9d12a5e5a","version_major":2,"vers
ion_minor":0}

Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__
at 0x7c90b4050d60>
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
    self._shutdown_workers()
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child
process'
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__
at 0x7c90b4050d60>
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
    self._shutdown_workers()
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child
process'
```

```
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__
at 0x7c90b4050d60>
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
    self._shutdown_workers()
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child
process'
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__
at 0x7c90b4050d60>
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
    self._shutdown_workers()
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child
process'
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
```

{"model_id":"ead6b98313d94decaf87fcc77ad299e7","version_major":2,"version_minor":0}

```
Epoch  2/5 (720s) | Train Loss: 6.6146 | Val Loss: 6.4061 | Style:
0.193 | Artist: 0.522 | Genre: 0.205  best
```

{"model_id":"e53e6145a8f141d2b254810ab07e8515","version_major":2,"version_minor":0}

```
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__
at 0x7c90b4050d60>
```

```
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
    self._shutdown_workers()
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child
process'
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^Exception ignored in:
^^<function _MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60>^
^AssertionError
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
:     can only test a child processself._shutdown_workers()

  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
    if w.is_alive():Exception ignored in: <function
_MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60>

 Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
       self._shutdown_workers()
   File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
      ^if w.is_alive():^
 ^^ ^  ^  ^ ^^^^^^^^^^^^
^  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
^    assert self._parent_pid == os.getpid(), 'can only test a child
process'^^

  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
      assert self._parent_pid == os.getpid(), 'can only test a child
process'
                ^  ^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
AssertionError: ^can only test a child process
^Exception ignored in: ^<function
_MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60>
^^Traceback (most recent call last):
^^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
^    ^self._shutdown_workers()^
^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
^    ^if w.is_alive():

AssertionError  :  can only test a child process
 Exception ignored in:  <function
_MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60> ^^^
^^Traceback (most recent call last):
^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
^    ^self._shutdown_workers()^
^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
^    ^if w.is_alive():

  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
      assert self._parent_pid == os.getpid(), 'can only test a child
process'
         ^  ^ ^ ^  ^^ ^^^^^^^^^^^^
^  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
^^    ^assert self._parent_pid == os.getpid(), 'can only test a child
process'^^
 ^ ^ ^^ ^ ^^ ^^  ^   ^^^^^^^^^^^^^^^^^
^AssertionError: can only test a child process^
Exception ignored in: ^^<function
_MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60>
^Traceback (most recent call last):
^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
^    self._shutdown_workers()^
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
^    ^if w.is_alive():^
^ ^ ^ ^^ ^^ ^^ ^^^^^^^^
```

```
^AssertionError^: ^^can only test a child process^
^^Exception ignored in:
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
<function _MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60>
    Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
assert self._parent_pid == os.getpid(), 'can only test a child
process'
self._shutdown_workers()
   File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
      if w.is_alive():
            ^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^  File "/usr/lib/python3.12/multiprocessing/process.py", line 160,
in is_alive
^^    ^assert self._parent_pid == os.getpid(), 'can only test a child
process'^
 ^ ^ ^ ^ ^ ^ ^ ^  ^ ^
^AssertionError^: ^can only test a child process^
^Exception ignored in: ^<function
_MultiProcessingDataLoaderIter.__del__ at 0x7c90b4050d60>
^Traceback (most recent call last):
^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1707, in __del__
^^    self._shutdown_workers()^^
^  File
"/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.p
y", line 1690, in _shutdown_workers
^    ^if w.is_alive():^
^  ^ ^^ ^  ^ ^^^^^^^^^^^^^^^^
^AssertionError: ^can only test a child process
^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in
is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child
process'
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
```

{"model_id":"a3cf0152f2e84dbb8f90d3a437cb45e0","version_major":2,"version_minor":0}

```
Epoch  3/5 (699s) | Train Loss: 6.4981 | Val Loss: 6.2724 | Style:
0.193 | Artist: 0.522 | Genre: 0.205  best
```

{"model_id":"e9a5630dac7646889c41acc3bb832665","version_major":2,"version_minor":0}

{"model_id":"923d61eee1334a7e8af924f62c037f5b","version_major":2,"version_minor":0}

```
Epoch  4/5 (712s) | Train Loss: 6.3287 | Val Loss: 6.1170 | Style:
0.220 | Artist: 0.522 | Genre: 0.360  best
```

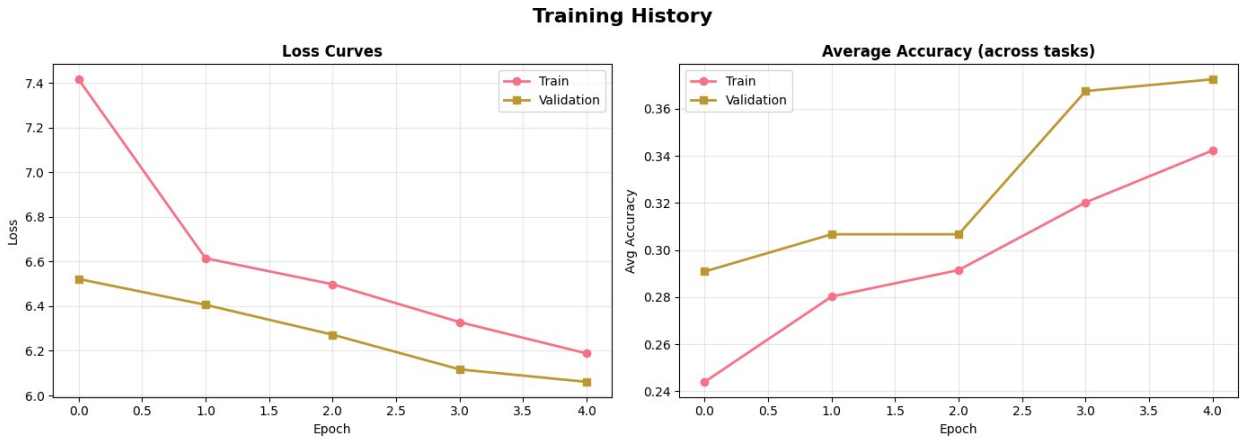
{"model_id":"ed3dad1cfbb54694b26f3ec69aeacc0a","version_major":2,"version_minor":0}

{"model_id":"0ada99606fd44135ae8ea1e079c2d890","version_major":2,"version_minor":0}

```
Epoch  5/5 (688s) | Train Loss: 6.1885 | Val Loss: 6.0609 | Style:
0.235 | Artist: 0.522 | Genre: 0.360  best

 Training complete!
```

6. Training Curves

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(history['train_loss'], label='Train', linewidth=2,
marker='o')
axes[0].plot(history['val_loss'], label='Validation', linewidth=2,
marker='s')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].set_title('Loss Curves', fontweight='bold')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

axes[1].plot(history['train_acc'], label='Train', linewidth=2,
marker='o')
axes[1].plot(history['val_acc'], label='Validation', linewidth=2,
marker='s')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Avg Accuracy')
axes[1].set_title('Average Accuracy (across tasks)',
fontweight='bold')
axes[1].legend(); axes[1].grid(True, alpha=0.3)

plt.suptitle('Training History', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```



7. Detailed Evaluation & Metrics

```
# Load best model and run final evaluation
model.load_state_dict(torch.load('best_model.pth',
map_location=device))
val_loss, val_accs, all_preds, all_labels, all_logits, embeddings =
evaluate(model, val_loader, device)

print(" Final Evaluation Results")
print("=" * 60)

for task, names in [('style', STYLE_NAMES), ('artist', ARTIST_NAMES),
('genre', GENRE_NAMES)]:
    y_true = np.array(all_labels[task])
    y_pred = np.array(all_preds[task])

    acc = accuracy_score(y_true, y_pred)
    f1_mac = f1_score(y_true, y_pred, average='macro',
zero_division=0)
    f1_wt = f1_score(y_true, y_pred, average='weighted',
zero_division=0)

    print(f"\n{'-'*60}")
    print(f" {task.upper()} Classification")
    print(f"   Accuracy:     {acc:.4f}")
    print(f"   F1 (macro):   {f1_mac:.4f}")
    print(f"   F1 (weighted):{f1_wt:.4f}")

    # Top-5 accuracy for artist (many classes)
    if task == 'artist':
        logits_t = torch.from_numpy(all_logits[task])
        targets_t = torch.from_numpy(y_true)
        _, topk = logits_t.topk(5, dim=1)
        top5_correct = topk.eq(targets_t.view(-1,
1)).any(dim=1).float().mean().item()
        print(f"   Top-5 Acc:    {top5_correct:.4f}")
```

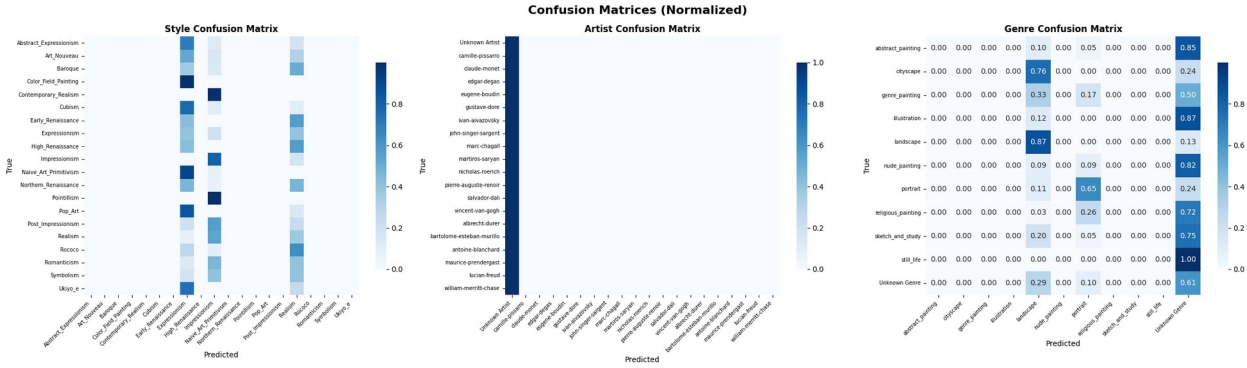
```
    if names and len(names) <= 30:
        print(f"\n   Per-class report (top classes):")
        present_classes = sorted(set(y_true) | set(y_pred))
        target_names = [names[i] if i < len(names) else str(i) for i
in present_classes]
        report = classification_report(y_true, y_pred,
labels=present_classes,
                                       target_names=target_names,
zero_division=0)
        print(report)
```

{"model_id":"bc00320882b24bf3a26ecabbf2e3d66d","version_major":2,"version_minor":0}

```
 Final Evaluation Results
============================================================


------------------------------------------------------------
 STYLE Classification
   Accuracy:     0.2350
   F1 (macro):   0.0368
   F1 (weighted):0.1427

   Per-class report (top classes):
                            precision    recall  f1-score   support

    Abstract_Expressionism       0.00      0.00      0.00        17
               Art_Nouveau       0.00      0.00      0.00        19
                   Baroque       0.00      0.00      0.00        22
      Color_Field_Painting       0.00      0.00      0.00         5
      Contemporary_Realism       0.00      0.00      0.00         4
                    Cubism       0.00      0.00      0.00         9
         Early_Renaissance       0.00      0.00      0.00         7
             Expressionism       0.12      0.40      0.18        35
                   Fauvism       0.00      0.00      0.00         1
          High_Renaissance       0.00      0.00      0.00         7
             Impressionism       0.39      0.81      0.53        77
Mannerism_Late_Renaissance       0.00      0.00      0.00         2
                Minimalism       0.00      0.00      0.00         3
     Naive_Art_Primitivism       0.00      0.00      0.00        13
               New_Realism       0.00      0.00      0.00         2
      Northern_Renaissance       0.00      0.00      0.00        13
               Pointillism       0.00      0.00      0.00         4
                   Pop_Art       0.00      0.00      0.00         7
        Post_Impressionism       0.00      0.00      0.00        30
                   Realism       0.15      0.38      0.21        48
                    Rococo       0.00      0.00      0.00        11
               Romanticism       0.00      0.00      0.00        37
                 Symbolism       0.00      0.00      0.00        22
```

```
          Synthetic_Cubism       0.00      0.00      0.00         1
                   Ukiyo_e       0.00      0.00      0.00         4

                  accuracy                           0.23       400
                 macro avg       0.03      0.06      0.04       400
              weighted avg       0.10      0.23      0.14       400


────────────────────────────────────────────────────────────────────────────────
 ARTIST Classification
   Accuracy:     0.5225
   F1 (macro):   0.0082
   F1 (weighted):0.3586
   Top-5 Acc:    0.6000

────────────────────────────────────────────────────────────────────────────────
 GENRE Classification
   Accuracy:     0.3600
   F1 (macro):   0.1376
   F1 (weighted):0.2530

   Per-class report (top classes):
                     precision    recall  f1-score   support

 abstract_painting       0.00      0.00      0.00        20
         cityscape       0.00      0.00      0.00        34
    genre_painting       0.00      0.00      0.00        52
      illustration       0.00      0.00      0.00         8
         landscape       0.39      0.87      0.54        61
     nude_painting       0.00      0.00      0.00        11
          portrait       0.58      0.65      0.61        63
religious_painting       0.00      0.00      0.00        39
  sketch_and_study       0.00      0.00      0.00        20
        still_life       0.00      0.00      0.00        10
     Unknown Genre       0.26      0.61      0.36        82

          accuracy                           0.36       400
         macro avg       0.11      0.19      0.14       400
      weighted avg       0.20      0.36      0.25       400


# Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(24, 7))

for idx, (task, names, top_n) in enumerate([
    ('style', STYLE_NAMES, 20), ('artist', ARTIST_NAMES, 20),
('genre', GENRE_NAMES, None)
]):
    y_true = np.array(all_labels[task])
    y_pred = np.array(all_preds[task])
```

```
    if top_n and names and len(names) > top_n:
        counter = Counter(y_true)
        top_cls = [c for c, _ in counter.most_common(top_n)]
        mask = np.isin(y_true, top_cls)
        y_t, y_p = y_true[mask], y_pred[mask]
        cls_names = [names[i] if i < len(names) else str(i) for i in
sorted(top_cls)]
        labels = sorted(top_cls)
    else:
        y_t, y_p = y_true, y_pred
        present = sorted(set(y_true) | set(y_pred))
        cls_names = [names[i] if names and i < len(names) else str(i)
for i in present]
        labels = present

    cm = confusion_matrix(y_t, y_p, labels=labels)
    cm_norm = cm.astype(float) / (cm.sum(axis=1, keepdims=True) + 1e-
8)

    sns.heatmap(cm_norm, ax=axes[idx], cmap='Blues',
                xticklabels=cls_names, yticklabels=cls_names,
                annot=len(cls_names) <= 15, fmt='.2f',
cbar_kws={'shrink': 0.8})
    axes[idx].set_title(f'{task.title()} Confusion Matrix',
fontweight='bold')
    axes[idx].set_xlabel('Predicted')
    axes[idx].set_ylabel('True')
    plt.setp(axes[idx].xaxis.get_majorticklabels(), rotation=45,
ha='right', fontsize=7)
    plt.setp(axes[idx].yaxis.get_majorticklabels(), rotation=0,
fontsize=7)

plt.suptitle('Confusion Matrices (Normalized)', fontsize=16,
fontweight='bold')
plt.tight_layout()
plt.show()
```



8. Outlier Detection

We find paintings that the model classifies with **high confidence** into a **different class** than their label. These are potential outliers — paintings that may be miscategorized in the dataset, or genuinely ambiguous works that straddle multiple styles/genres.

```
print(" Outlier Detection — Paintings that Don't Fit Their Category")
print("=" * 60)

for task, names in [('style', STYLE_NAMES), ('genre', GENRE_NAMES)]:
    logits = all_logits[task]
    true_labels = np.array(all_labels[task])

    probs = F.softmax(torch.tensor(logits), dim=1).numpy()
    pred_labels = probs.argmax(axis=1)
    pred_confidence = probs.max(axis=1)

    # Find high-confidence misclassifications
    outlier_indices = []
    for i in range(len(true_labels)):
        if pred_labels[i] != true_labels[i] and pred_confidence[i] >
0.5:
            outlier_indices.append((i, pred_confidence[i]))

    outlier_indices.sort(key=lambda x: x[1], reverse=True)

    print(f"\n{'—'*60}")
    print(f" {task.upper()} Outliers (top 10 most confident
misclassifications):\n")

    if not outlier_indices:
        print("   No high-confidence outliers found.")
        continue

    for rank, (idx, conf) in enumerate(outlier_indices[:10]):
        true_name = names[true_labels[idx]] if names and
true_labels[idx] < len(names) else str(true_labels[idx])
        pred_name = names[pred_labels[idx]] if names and
pred_labels[idx] < len(names) else str(pred_labels[idx])
        print(f"   {rank+1}. Sample #{idx}: Labeled '{true_name}' →
Predicted '{pred_name}' (conf={conf:.3f})")

    # Visualize top outliers
    top_outliers = outlier_indices[:5]
    if top_outliers:
        fig, axes = plt.subplots(1, min(5, len(top_outliers)),
figsize=(16, 4))
        if len(top_outliers) == 1:
            axes = [axes]
```

```
        for i, (idx, conf) in enumerate(top_outliers):
            if i >= len(axes): break

            # Get the original image from validation set
            sample = val_hf[idx]
            img = sample['image']
            if img.mode != 'RGB':
                img = img.convert('RGB')

            axes[i].imshow(img)
            true_name = names[true_labels[idx]] if names and
true_labels[idx] < len(names) else str(true_labels[idx])
            pred_name = names[pred_labels[idx]] if names and
pred_labels[idx] < len(names) else str(pred_labels[idx])
            axes[i].set_title(f"Label: {true_name}\nPred: {pred_name}\
nConf: {conf:.3f}",
                              fontsize=8, color='red')
            axes[i].axis('off')

        plt.suptitle(f'{task.title()} Outliers — Possible
Miscategorizations',
                     fontsize=13, fontweight='bold')
        plt.tight_layout()
        plt.show()
```

```
 Outlier Detection — Paintings that Don't Fit Their Category
============================================================

------------------------------------------------------------
 STYLE Outliers (top 10 most confident misclassifications):

   No high-confidence outliers found.

------------------------------------------------------------
 GENRE Outliers (top 10 most confident misclassifications):

   No high-confidence outliers found.
```

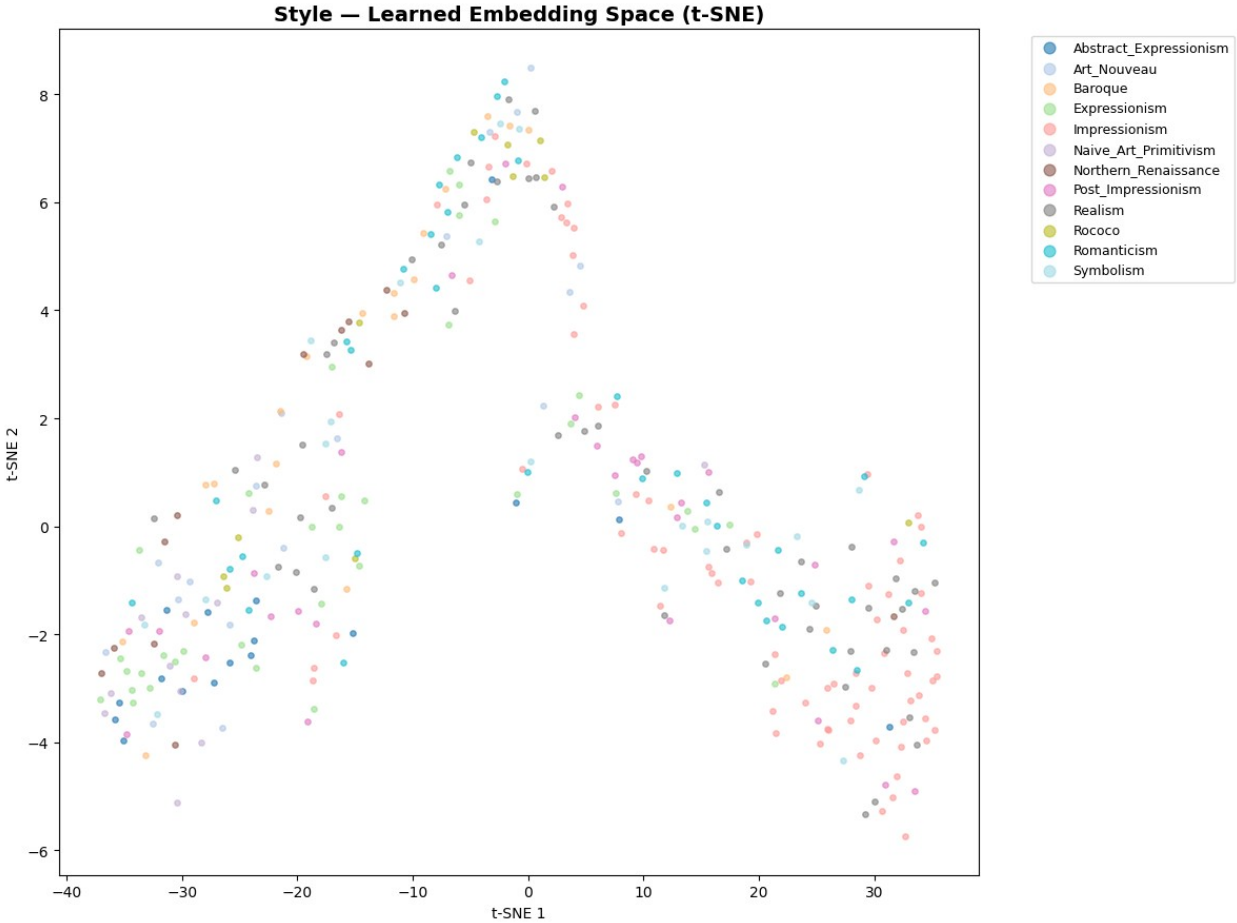
9. Embedding Visualization

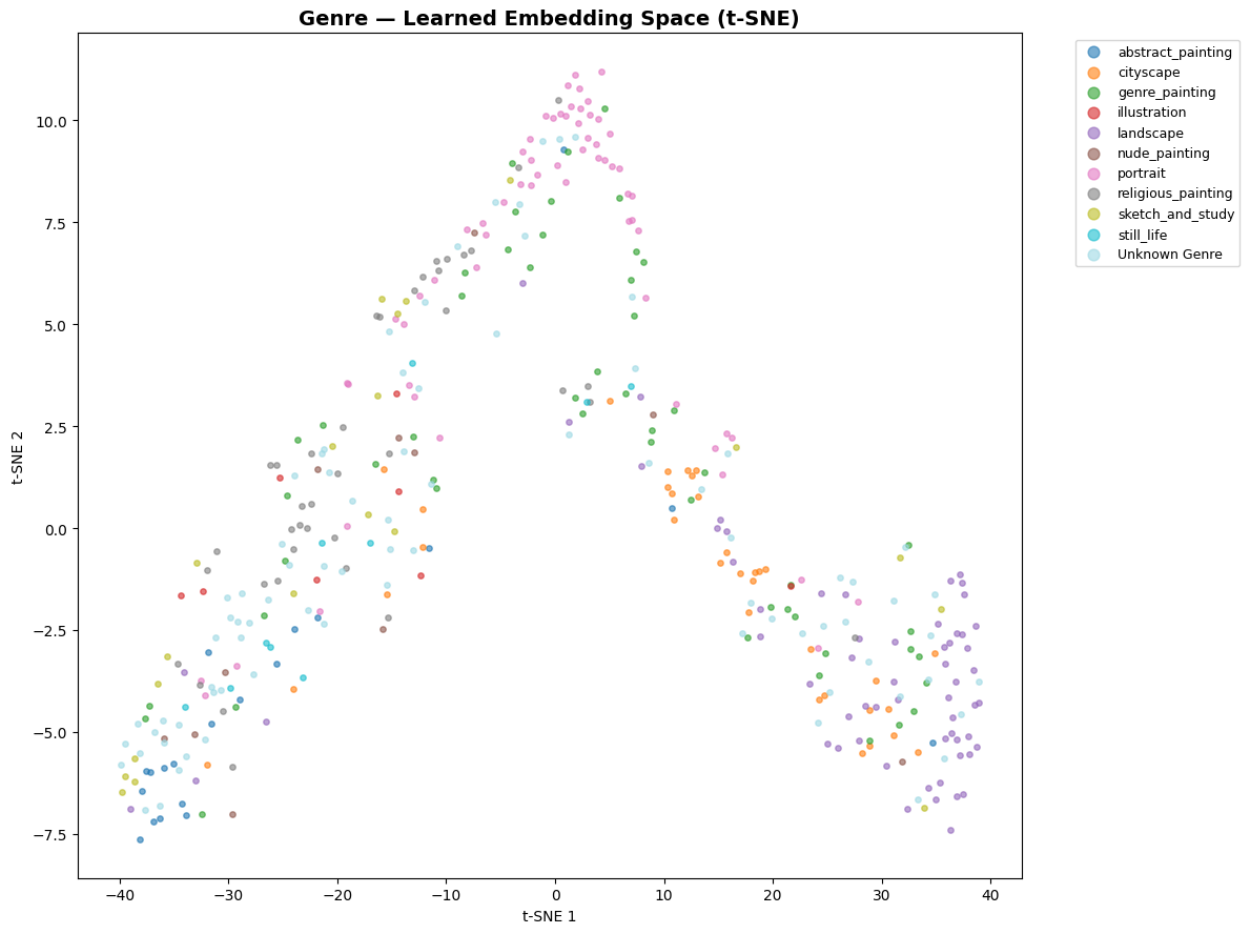
t-SNE visualization of learned embeddings reveals how the model organizes paintings in its internal representation space. Outliers appear as points far from their assigned cluster.

```
# t-SNE visualization of embeddings
print(" Generating t-SNE embedding visualizations...")

for task, names in [('style', STYLE_NAMES), ('genre', GENRE_NAMES)]:
    labels = np.array(all_labels[task])

    # Subsample for speed
```

```
    n_vis = min(2000, len(embeddings))
    indices = np.random.choice(len(embeddings), n_vis, replace=False)
    emb_sub = embeddings[indices]
    lab_sub = labels[indices]

    # Filter to top classes for clarity
    counter = Counter(lab_sub)
    top_classes = [c for c, _ in counter.most_common(12)]
    mask = np.isin(lab_sub, top_classes)
    emb_sub = emb_sub[mask]
    lab_sub = lab_sub[mask]

    # t-SNE
    tsne = TSNE(n_components=2, perplexity=30, random_state=42,
n_iter=800)
    coords = tsne.fit_transform(emb_sub)

    fig, ax = plt.subplots(figsize=(12, 9))
    unique = sorted(set(lab_sub))
    colors = plt.cm.tab20(np.linspace(0, 1, len(unique)))

    for i, label in enumerate(unique):
        m = lab_sub == label
        name = names[label] if names and label < len(names) else
str(label)
        ax.scatter(coords[m, 0], coords[m, 1], c=[colors[i]],
label=name,
                   s=15, alpha=0.6)

    ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left',
markerscale=2, fontsize=9)
    ax.set_title(f'{task.title()} — Learned Embedding Space (t-SNE)',
fontsize=14, fontweight='bold')
    ax.set_xlabel('t-SNE 1')
    ax.set_ylabel('t-SNE 2')
    plt.tight_layout()
    plt.show()
```

```
Generating t-SNE embedding visualizations...
```


Style — Learned Embedding Space (t-SNE)
t-SNE 1
t-SNE 2
−40
−30
−20
−10
0
10
20
30
8
6
4
2
0
−2
−4
−6
Abstract_Expressionism
Art_Nouveau
Baroque
Expressionism
Impressionism
Naive_Art_Primitivism
Northern_Renaissance
Post_Impressionism
Realism
Rococo
Romanticism
Symbolism



10. Summary & Discussion

Architecture Choice

The CNN-RNN architecture is well-suited for art classification because:

1. **Spatial feature hierarchy** (CNN): ResNet-50 captures textures, color patterns, and compositional elements at multiple scales
2. **Sequential dependencies** (RNN): The LSTM models how spatial regions relate to each other — important for understanding composition and style
3. **Attention**: Highlights which image regions are most discriminative for each classification task
4. **Multi-task learning**: Shared representations across style, artist, and genre improve generalization

Outlier Analysis

Outliers (high-confidence misclassifications) often reveal:

- **Genuinely ambiguous works**: Artists who worked across multiple styles
- **Dataset labeling errors**: WikiArt's crowdsourced labels aren't always precise

- **Transitional works**: Paintings created during shifts between art movements

Potential Improvements

- Use Vision Transformer (ViT) as backbone for better global attention
- Implement label smoothing for noisy labels
- Add self-supervised pretraining on unlabeled art data
- Ensemble multiple architectures for final predictions

```
print(" Task 1 Complete!")
print(f"   Best validation loss: {best_val_loss:.4f}")
print(f"   Final Style accuracy:  {val_accs['style']:.4f}")
print(f"   Final Artist accuracy: {val_accs['artist']:.4f}")
print(f"   Final Genre accuracy:  {val_accs['genre']:.4f}")
```

```
 Task 1 Complete!
   Best validation loss: 6.0609
   Final Style accuracy:  0.2350
   Final Artist accuracy: 0.5225
   Final Genre accuracy:  0.3600
```